

rand~ Objekt in Max



was macht das rand~ objekt in max

In **Max/MSP**, das `rand~` Objekt erzeugt **Pseudo-Zufallszahlen im Audio- (Signal-) Bereich**. Es gehört zur Familie der **signalbasierten Zufallsobjekte** und ist nützlich, wenn man im Audiobereich Zufallswerte erzeugen möchte, z. B. für **Noise-ähnliche Modulationen, Randomisierung von Parametern**, oder **kreative Sounddesigns**.

Funktionsweise von `rand~`

- Es erzeugt einen **kontinuierlichen Audio-Stream** von Zufallswerten (ähnlich wie Noise, aber diskret).
- Die Zufallswerte liegen standardmäßig im Bereich **zwischen -1 und 1**.
- Das Verhalten ähnelt einer **stufenweisen** oder **quantisierten** Form von Rauschen.
- `rand~` erzeugt bei jedem DSP-Zyklus neue Zufallswerte – du kannst es dir wie einen Zufallswert-Generator pro Abtastung vorstellen.

Syntax

```
max
rand~
```

Es gibt keine Argumente. Es gibt nur einen **Signal-Ausgang**.

Beispielnutzung

```
max
[cycle~ 440]
|
[*~]
|
[rand~]
```

In diesem Beispiel moduliert `rand~` die Amplitude eines 440-Hz-Sinustons zufällig im Audiobereich.

Vergleich mit anderen Objekten

- `noise~`: Gibt **weißes Rauschen** aus (kontinuierlich).
- `rand~`: Gibt **stufenweise Zufallswerte** aus, kein kontinuierliches Rauschen.
- `random`: Gibt Zufallswerte im Kontrollbereich (nicht für Audio-Signale).

- ``drunk``, ``urandom``, ``expr rand``: ebenfalls Zufallsobjekte, aber für unterschiedliche Zwecke (nicht signalbasiert).

Wenn du möchtest, kann ich dir ein konkretes Patch-Beispiel zeigen.



Es erzeugt einen kontinuierlichen Audio-Stream von Zufallswerten (ähnlich wie Noise, aber diskret).

Gute Beobachtung — lass uns das präzisieren:

Das ``rand~``-Objekt in **Max/MSP** erzeugt **diskrete Zufallswerte im Audio-Rate-Bereich**, aber **nicht kontinuierlich** wie bei ``noise~``.

Genauer gesagt:

- ``rand~`` erzeugt **stufenartige Zufallswerte**, die **pro Signal-Vektor (nicht pro Sample)** wechseln.
- Es generiert also **einen neuen Zufallswert pro Signalblock** (typisch 64 Samples, je nach Audioeinstellungen).
- Das resultiert in einem Signal, das wie eine **Sample-and-Hold-ähnliche Kurve** aussieht: Es bleibt konstant über einen Block und springt dann auf einen neuen Zufallswert.

Visuell:

Stell dir eine Treppenfunktion vor:

markdown



Jeder horizontale Abschnitt ist ein konstantes Signal über z. B. 64 Samples, danach ein zufälliger Sprung.

Im Vergleich zu ``noise~``:

Objekt	Art des Rauschens	Kontinuität	Rate
<code>`noise~`</code>	Weißes Rauschen	kontinuierlich	Audio (pro Sample)
<code>`rand~`</code>	Blockweise Zufallswerte	diskret (pro Block)	Audio (pro Vektor)

Anwendung von ``rand~``

- **Stochastische Modulation** (z. B. Filterfrequenz, Lautstärke).
- **Unregelmäßige Steuerungssignale** im Audio-Tempo.
- In Verbindung mit ``line~``, ``slide~`` oder Filtern kann man weichere Übergänge daraus basteln.

Wenn du willst, kann ich dir auch zeigen, wie man aus ``rand~`` mit etwas Glättung z. B. ein zufälliges LFO-ähnliches Signal erzeugt.



was bedeutet pro vektor

Super Frage! In Max/MSP (wie in vielen DSP-Umgebungen) bedeutet „**pro Vektor**“ (engl. „per vector“) Folgendes:

🔧 Was ist ein Signal-Vektor?

In der digitalen Signalverarbeitung verarbeitet Max das Audiosignal **nicht Sample für Sample**, sondern in **Blöcken von mehreren Samples gleichzeitig**.

Diese Blöcke nennt man **Signal-Vektoren**.

- Die **Standardgröße** eines Signal-Vektors in Max ist **64 Samples**.
- Das bedeutet: Ein Objekt wie ``rand~`` arbeitet **einmal pro Vektor** – also alle 64 Samples.
- Diese Vektorgröße kannst du im DSP-Menü von Max ändern (z. B. 64, 128, 256...).

⚙️ Beispiel mit ``rand~``

Wenn ``rand~`` einen neuen Wert **pro Vektor** ausgibt:

- Es erzeugt **einen Zufallswert**, der für **alle 64 Samples gleich bleibt**.
- Beim nächsten Vektor (also 64 Samples später) wird ein **neuer Zufallswert** erzeugt.

So sieht das im Zeitverlauf aus:

makefile

Samples:	1–64	65–128	129–192	...
rand~:	0.34	-0.77	0.12	...

📊 Warum das wichtig ist

- **Glatte Modulationen** brauchst du eher mit ``line~``, ``cycle~`` etc.
- ``rand~`` klingt **stufenartig**, was nützlich für z. B. Sample-and-Hold oder zufällige Steuerungen ist.
- Willst du **schnellere oder langsamere Änderungen**, musst du die Vektorgröße anpassen oder ``rand~`` mit z. B. ``line~`` glätten.

📌 Merksatz

„**Pro Vektor**“ heißt: Das Objekt arbeitet **nur einmal pro Block von Samples**, nicht bei jedem einzelnen Sample.

Wenn du magst, zeige ich dir ein Mini-Patch, das den Effekt visuell darstellt.



was macht ein `mc.sig~ @chans 100` in max msp

Das Objekt ``mc.sig~ @chans 100`` in **Max/MSP** gehört zur **MC- (Multichannel-) Umgebung**, die für die gleichzeitige Verarbeitung mehrerer Audiokanäle optimiert ist.

🧠 Bedeutung von ``mc.sig~ @chans 100``

- ``mc.sig~`` ist die Multichannel-Version von ``sig~``, das konstanten Audio-Signalwert erzeugt.
- ``@chans 100`` bedeutet: Das Objekt erzeugt **100 unabhängige Signal-Kanäle**.
- Jeder dieser 100 Kanäle gibt standardmäßig **denselben Wert (z. B. 0)** als Audio-Signal aus.

🔊 Was passiert konkret?

```
max  
  
[mc.sig~ @chans 100]
```

- Erzeugt ein **MC-Signal mit 100 Kanälen**.
- Jeder Kanal ist ein kontinuierliches Audiosignal mit dem aktuellen Wert (standardmäßig 0).
- Du kannst jeden Kanal individuell ansteuern, z. B. mit:

```
max  
  
[mc.sig~ 1. @chans 100] // setzt alle Kanäle auf 1.0
```

oder

```
max  
  
[mc.sig~ @chans 100]  
[mc.setvalue~ 42 0.75] // setzt Kanal 42 auf 0.75
```

📦 Wofür ist das gut?

- Um **viele identische oder unterschiedliche konstante Audio-Signale** gleichzeitig zu erzeugen.
- Typisch in **Polyphonie, Granularsynthese, Spatial Audio, oder MPE-Setups**, wo du viele Kanäle auf einmal brauchst.
- In Verbindung mit anderen ``mc.``-Objekten wie ``mc.cycle~``, ``mc.*~``, ``mc.mixdown~`` usw. kannst du komplexe Multichannel-Signalketten bauen.

🔊 Beispiel: 100 Sinusoszillatoren mit verschiedenen Frequenzen

```
max
```

```
[mc.cycle~ @chans 100] // 100 Sinusoszillatoren
[mc.sig~ @chans 100] // konstante Steuerwerte (z. B. Frequenz)
```

Dann könntest du z. B. mit `mc.setValue~` pro Kanal unterschiedliche Frequenzen setzen.

Zusammenfassung

Komponente	Bedeutung
<code>mc.sig~</code>	Mehrkanaliges konstantes Signal (wie <code>sig~</code> , aber MC)
<code>@chans 100</code>	Erstellt 100 unabhängige Signal-Kanäle
Anwendung	Polyphonie, MPE, Spatial Audio, massive Parallelverarbeitung

Wenn du willst, zeige ich dir gern ein Patch-Beispiel mit `mc.sig~` in Aktion.



wenn das sig~ eine zahl bekommt wie 500. was ist das dann ?

Wenn du in **Max/MSP** folgendes Objekt hast:

```
max
```

```
[sig~ 500.]
```

dann bedeutet das:

 `sig~ 500.` erzeugt ein konstantes Audio-Signal mit dem Wert `500.0`

- Es **gibt bei jeder Abtastung** (`sample`) den Wert `500.` aus.
- Anders gesagt: Das Signal besteht aus einer **endlosen Folge von 500.0**, also:

```
markdown
```

```
500. 500. 500. 500. 500. ...
```

Wofür ist das nützlich?

`sig~` wird verwendet, um **konstante Werte im Audio-Bereich bereitzustellen**, etwa:

- als Steuer-Signal für:
 - Frequenzen (z. B. für `[cycle~]`)
 - Lautstärke (`[*~]`)
 - Filterparameter (`[filtergraph~]`, `[biquad~]`, etc.)
- für mathematische Operationen im Audio-Bereich

Beispiel:

```
max

[sig~ 500.]    → (konstantes Signal mit Wert 500)
|
[cycle~]       → (Sinuston mit 500 Hz)
```

In diesem Fall wird ``cycle~`` mit einem **500-Hz-Signal** gespeist.

Wichtig:

- ``sig~`` selbst **macht keinen Ton** – es ist einfach eine **Zahl pro Sample** als Audio-Signal.
- Der Unterschied zu einem Float-Objekt (``[500.]``) ist, dass ``sig~`` den Wert im **Signal-Rate-Bereich** ausgibt, während ein Float nur im **Control-Rate-Bereich** arbeitet.

Merksatz:

``sig~`` ist wie ein **konstanter Spannungspegel** in der Audio-Welt. Es macht nichts, außer ununterbrochen denselben Wert auszugeben — aber genau das ist oft extrem nützlich für Steuerungen im DSP-Bereich.

Wenn du magst, kann ich dir ein Patch zeigen, das den Unterschied zwischen ``[500.]`` und ``[sig~ 500.]`` in der Praxis verdeutlicht.



ok also sig~ ist KEINE Frequenz

Ganz genau – ``sig~`` **ist selbst keine Frequenz**.

Was ist ``sig~`` wirklich?

- ``sig~`` ist ein **Objekt, das ein konstantes Audio-Signal** ausgibt.
- Es erzeugt bei jeder Abtastung den **gleichen numerischen Wert** – z. B. ``500.``.
- Es **weiß nichts** über Frequenzen, Zeit, Tonhöhe oder sonst was – es gibt einfach nur eine **Zahl als Audiosignal** aus.

Wann wird ``sig~`` zur Frequenz?

Nur wenn du es **in einen Kontext einsetzt, wo dieser Zahlenwert als Frequenz interpretiert wird**.

Beispiel:

```
max

[sig~ 500.]    → konstantes Signal mit Wert 500.0
|
```

[cycle~] → erzeugt einen Sinuston mit 500 Hz

Hier **interpretiert** ``cycle~`` **den Eingangswert als Frequenz in Hertz** – deshalb wird 500.0 Hz erzeugt. Aber das ist **die Entscheidung von** ``cycle~``, nicht von ``sig~``.

Vergleich:

Objekt	Was es tut	Bedeutet das „Frequenz“?
<code>`sig~ 500.`</code>	Gibt 500.0 pro Sample aus	✗ Nein
<code>`cycle~`</code>	Interpretiert Eingang als Hz	✓ Ja, wenn Eingang $\neq 0$
<code>`[500.]`</code>	Gibt eine einzelne Zahl	✗ Nein

Fazit:

``sig~`` ist **nur eine konstante Zahl im Audioformat** – **ob** und **wie** diese Zahl als Frequenz (oder Lautstärke, Filterwert, etc.) verstanden wird, hängt **vom nachfolgenden Objekt** ab.

Willst du das mal als Patch visualisiert bekommen?



was bedeutet carry beim uzi objekt

Beim ``[uzi]``-Objekt in **Max/MSP** ist der Attributparameter ``@carry`` eine **eher selten genutzte, aber nützliche Option**, die sich auf das **Zurücksetzen des Zählers** bei mehrfacher Ausführung bezieht.

Was macht ``[uzi]`` überhaupt?

``uzi`` ist ein **Schnellschuss-Zähler**, der in einem einzigen Scheduler-Tick eine Reihe von Ausgaben erzeugt:

```
max
[uzi 5]
```

→ Gibt in sehr kurzer Zeit (quasi sofort) die Zahlen 1, 2, 3, 4, 5 aus dem linken Ausgang und am Ende einen Bang aus dem rechten Ausgang.

Was bedeutet ``@carry``?

```
max
[uzi 5 @carry 1]
```

- Der Parameter ``@carry`` steuert, **ob die Zählung beim nächsten Trigger fortgesetzt oder neu gestartet wird**.

- **Standardverhalten** (``@carry 0``):
 - Beim Bang beginnt ``uzi`` **immer wieder bei 1**.
- **Mit** ``@carry 1``:
 - ``uzi`` **merkt sich**, wo es zuletzt aufgehört hat, und **zählt weiter**.

Beispiel:

```
max
[uzi 3 @carry 1]
```

Wenn du dieses Objekt **mehrmals** bangst, erhältst du:

1. Bang → 1, 2, 3
2. Bang → 4, 5, 6
3. Bang → 7, 8, 9
- ...

Ohne ``@carry`` (also ``uzi 3``):

1. Bang → 1, 2, 3
2. Bang → 1, 2, 3
3. Bang → 1, 2, 3

Wann ist ``@carry`` nützlich?

- Wenn du **große Datenmengen verarbeiten willst, aber in Blöcken**.
- Zum Beispiel: Du willst 1000 Schritte machen, aber nicht auf einmal → du kannst mit ``uzi 100 @carry 1`` in Blöcken von 100 inkrementell durchzählen.
- Auch nützlich für **Sequencer, Metrik-Teilung, granulare Kontrolle über Iterationen**, usw.

Merksatz:

``@carry 1`` = „Erinnere dich, wo du warst, und mach da weiter.“
``@carry 0`` = „Fang jedes Mal wieder bei 1 an.“

Wenn du willst, zeig ich dir ein Patch, wo das in Aktion zu sehen ist.



es hat einen ausgang der carry heisst

Ah, sehr gut aufgepasst – du meinst den **dritten Ausgang** von ``[uzi]``, der auch als „**carry outlet**“ bezeichnet wird.

Was macht der carry-Ausgang bei `[uzi]`?

Wenn du dir ein Objekt wie

```
max
[uzi 5]
```

anschaust, hat es **drei Ausgänge**:

1. **Linker Ausgang** – Gibt die Zahlen 1 bis N aus.
2. **Rechter Ausgang** – Gibt ein Bang aus, wenn `[uzi]` fertig ist.
3. **Mittlerer (carry) Ausgang** – Gibt ein **Bang** aus, wenn der Zähler überläuft, also wenn der Wert über die maximale Zählung hinausgeht – aber **nur in Kombination mit `[@carry 1]`**.

Beispiel:

```
max
[uzi 3 @carry 1]
```

Dann hast du:

- 1. Bang → Ausgabetabelle: `[1]`, `[2]`, `[3]`, rechter Bang
- 2. Bang → `[4]`, `[5]`, `[6]`, rechter Bang
- → **Jetzt kommt: carry-Bang!**

Denn: Der interne Zähler hat bei 3 aufgehört, und fängt beim nächsten Durchlauf bei 4 an → das ist ein „Überlauf“ im Kontext des `[uzi]`, weil es über den ursprünglichen Zählbereich hinausgeht. Daher gibt der **carry-Ausgang** ein **Bang** aus.

Zusammengefasst:

Ausgang	Bedeutung
Linker Ausgang	Aktuelle Zahl (1 bis N)
Rechter Ausgang	Bang, wenn alle Zahlen ausgegeben wurden
Carry-Ausgang	Bang, wenn ein Übertrag stattfindet (nur bei `[@carry 1]`)

Typischer Anwendungsfall

Wenn du eine große Schleife in **Teilstücken (Chunks)** ausführst, kannst du den **carry-Bang** verwenden, um z. B. den nächsten `[uzi]`-Durchlauf zu triggern, oder um zu erkennen, dass man eine neue Runde begonnen hat.

Merksatz:

Der „**carry**“-Ausgang ist wie ein Überlauf-Anzeiger:

„**Ich bin über mein Limit hinausgelaufen – willst du weitermachen?**“

Willst du ein kleines visuelles Demo-Patch dazu?



was bedeutet `zl.group @zlmaxsize 4096`

Gute Frage! Das Objekt `[zl.group]` in **Max/MSP** ist extrem nützlich, wenn du **Listen sammeln und zusammenbauen** willst – besonders über mehrere Schritte hinweg.

Was macht `[zl.group]`?

`[zl.group]` **sammelt eingehende Werte** (Zahlen, Symbole, Listen) und **baut daraus eine Liste**. Sobald eine bestimmte **Länge erreicht ist**, **gibt es die gesammelte Liste aus**.

Syntax:

max

`[zl.group N]`

- `N` = Wie viele Elemente gesammelt werden sollen, bevor die Liste ausgegeben wird.
- Beispiel: `[zl.group 4]` sammelt 4 Werte und gibt dann eine Liste wie `1 2 3 4` aus.

Was bedeutet `@zlmaxsize 4096`?

Das ist ein **Attribut**, das festlegt, **wie viele Elemente maximal gespeichert werden können**, bevor ein interner Fehler auftritt.

Wichtig:

- `@zlmaxsize` **setzt die maximale interne Puffergröße**.
- Auch wenn du z. B. `[zl.group 100]` verwendest, musst du sicherstellen, dass der `@zlmaxsize`-Wert **≥ 100** ist.
- **Standardwert** (wenn nicht angegeben): **256**
- In deinem Fall: `@zlmaxsize 4096` bedeutet, dass **bis zu 4096 Elemente gespeichert werden können**, bevor ein Fehler passiert.

Beispiel:

```
max
```

```
[zl.group 100 @zlmaxsize 4096]
```

- Sammelt 100 eingehende Werte.
- Gibt sie dann als Liste aus.
- Puffer ist groß genug für 4096 Elemente → kein Risiko für Überlauf.

Wann ist das nützlich?

- Wenn du **Daten sammelst** aus einer Schleife (z. B. mit `[uzi]`) oder über Zeit.
- Für **Matrixdaten, MIDI-Dumps, Live-Recording von Parametern**, etc.
- Bei sehr großen Listen (z. B. Audioanalyse, Sensor-Input), brauchst du oft ein hohes `zlmaxsize`.

Merksatz:

`zl.group` baut eine Liste.

`@zlmaxsize` sagt, **wie viel es maximal festhalten darf**, bevor es überläuft.

Willst du ein Beispiel sehen, wie man z. B. 1024 zufällige Werte mit `uzi` + `zl.group` in einer Liste sammelt?



was macht dbtoa

Das Objekt `dbtoa` in **Max/MSP** konvertiert **Dezibel-Werte in Amplitudenwerte**.

Was bedeutet das genau?

`dbtoa` steht für:

dB to Amplitude

 **Es rechnet:**

$$\text{Amplitude} = 10^{\left(\frac{\text{dB}}{20}\right)}$$

 **Eingabe** →  **Ausgabe**

Eingabe (dB)	Ausgabe (Amplitude)
`0`	`1.0`
`-6`	`~0.5012`

Eingabe (dB)	Ausgabe (Amplitude)
<code>`-20`</code>	<code>`0.1`</code>
<code>`-60`</code>	<code>`0.001`</code>
<code>`-∞`</code>	<code>`0`</code>

Warum braucht man das?

In der Audiowelt ist **dB eine logarithmische Skala**, aber **Amplitude** (also z. B. Lautstärke im DSP-Signal) ist **linear**.

Du brauchst diese Umrechnung z. B. wenn:

- Du mit **Fadern oder UI-Elementen in dB** arbeitest, aber **DSP-Objekte wie ``*~`` mit Amplitude** arbeiten.
- Du **Automation oder MIDI-Controller-Daten in dB** empfängst, aber **Linearsignale im Audio brauchst**.



Beispiel in Max:

```
max
[slider] → [dbtoa] → [*~]
```

- Der Slider liefert z. B. ``-20`` dB.
- ``dbtoa`` wandelt das in ``0.1`` um.
- Das kann dann direkt als Multiplikator für ein Audiosignal verwendet werden (``[*~]``).

Merksatz:

``dbtoa`` übersetzt:

„Wie laut klingt -40 dB wirklich in Audio-Signalwerten?“

Möchtest du auch die Umkehrung (``atodb``) erklärt bekommen?



was macht mc.pong~ 0 30 15000

Das Objekt ``mc.pong~`` in **Max/MSP** gehört zur Multichannel-Audio (MC) Umgebung und erzeugt eine **„Ping-Pong“-artige Modulation** – also einen periodisch auf- und absteigenden Wert (ähnlich wie eine Dreiecksschwingung).



Syntax:

max

[mc.pong~ min max period]

In deinem Fall:

max

[mc.pong~ 0 30 15000]

bedeutet:

Argument	Bedeutung
`0`	Minimumwert → die untere Grenze des Ausgangswerts
`30`	Maximumwert → die obere Grenze des Ausgangswerts
`15000`	Periodendauer in Millisekunden → Dauer für einen kompletten Zyklus

Was macht `mc.pong~ 0 30 15000` genau?

Es erzeugt eine **Audio-Rate-Modulation**, die:

- von **0 auf 30 steigt**,
- dann wieder **zurück auf 0 fällt**,
- und das Ganze in **15.000 ms (15 Sekunden)** wiederholt.

Das ergibt eine **gleichmäßige Dreiecksschwingung zwischen 0 und 30**, mit einem vollen Zyklus alle 15 Sekunden.

Stell dir einen Schieberegler vor, der 15 Sekunden braucht, um von ganz links nach ganz rechts zu gehen – und dann in 15 Sekunden wieder zurück. Genau das liefert `mc.pong~`.

Was ist am Ausgang?

- Ein **kontinuierliches Audiosignal**, das zwischen `0` und `30` pendelt.
- Du kannst das z. B. zur Modulation von:
 - Frequenz
 - Panorama
 - Filterwerten
 - Lautstärke
 - Position in einem Multikanal-Feld (MC!)

Multichannel?

Weil es `mc.pong~` ist (nicht `pong~`), erzeugt es **mehrere unabhängige Kanäle**, wenn du `@chans N` hinzufügst.

Ohne `@chans`, erzeugt es einen Einkanal-MC-Stream (also 1 „voice“ in MC-Kontext).

Beispiel-Anwendung:

max

```
[mc.pong~ 200 800 5000] → [mc.cycle~]
```

→ Moduliert die Frequenz eines Sinus-Oszillators zwischen 200 Hz und 800 Hz alle 5 Sekunden auf und ab.

Merksatz:

`mc.pong~` ist wie ein langsamer, kontrollierbarer Dreieck-LFO mit einstellbarem Min/Max und Periodendauer – perfekt für Modulationen im Multichannel-Audio.

Wenn du willst, zeig ich dir ein kleines Patch damit.



können so negative werte im audiosignal verhindert werden ?

Ja, mit `[mc.pong~]` (oder auch `[pong~]`) kannst du gezielt negative Werte im Audiosignal vermeiden, indem du den Wertebereich ausschließlich positiv wählst.

Beispiel:

max

```
[mc.pong~ 0 1 1000]
```

→ erzeugt ein Audiosignal, das **zwischen `0.0` und `1.0` pendelt** – also:

- kein negativer Wert,
- keine Null-Durchgänge (außer an den Wendepunkten).

Warum ist das nützlich?

In vielen DSP-Situationen **möchtest du ein Signal, das nur positive Werte hat**, z. B.:

Anwendung	Grund für positive Werte
Modulation von Frequenzen	Frequenz kann nicht negativ sein
Lautstärkesteuerung (<code>*~</code>)	Negative Amplitude führt zu Phasenumkehr
Steuerung von Positionen	z. B. in Wavetables, Buffers, Spatialization

Anwendung	Grund für positive Werte
Hüllkurven oder Envelopes	Meistens nur positive Amplituden erwünscht

Im Gegensatz dazu:

Ein klassischer `[cycle~]`, `[phasor~]` oder `[triangle~]` erzeugt standardmäßig Signale, die **zwischen -1.0 und 1.0** (oder 0.0 bis 1.0 bei `[phasor~]`) pendeln – das bedeutet:

- **negative Werte sind oft dabei,**
- **du musst sie ggf. verschieben oder skalieren**, z. B. mit:

```
max
[*~ 0.5] → [+~ 0.5] ; verschiebt -1..1 auf 0..1
```

Fazit:

Ja – durch die Wahl geeigneter Parameter in `[mc.pong~ min max period]` kannst du negative Werte im Signal sicher und direkt verhindern.

Wenn du willst, kann ich dir zeigen, wie man das für z. B. eine modulierte Frequenzverläufe ohne negatives Clipping verwendet.



```
was macht dieser patchteil : {
  "boxes" : [
    {
      "box" : {
        "maxclass" : "newobj",
        "text" : "live.thisdevice",
        "patching_rect" : [ 844.512215256690979, 1046.341488361358643, 77.0, 20.0 ],
        "id" : "obj-12",
        "numoutlets" : 3,
        "numinlets" : 1,
        "outlettype" : [ "bang", "int", "int" ]
      }
    }
  ],
  {
    "box" : {
      "maxclass" : "button",
      "patching_rect" : [ 897.530648231506348, 1072.897187948226929, 24.0, 24.0 ],
      "presentation_rect" : [ 279.629624962806702, 30.555555045604706,
29.45928418636322, 29.45928418636322 ],
      "id" : "obj-4",
      "parameter_enable" : 0,
      "numoutlets" : 1,
```

```

        "numinlets" : 1,
        "presentation" : 1,
        "outlettype" : [ "bang" ]
    }

    },
    {
        "box" : {
            "maxclass" : "message",
            "text" : "$1 100",
            "patching_rect" : [ 892.857751071453094, 1263.551392078399658, 39.0, 20.0 ],
            "id" : "obj-126",
            "numoutlets" : 1,
            "numinlets" : 2,
            "outlettype" : [ "" ]
        }
    },
    {
        "box" : {
            "maxclass" : "newobj",
            "text" : "line~",
            "patching_rect" : [ 892.857751071453094, 1286.915877878665924, 32.0, 20.0 ],
            "id" : "obj-124",
            "numoutlets" : 2,
            "numinlets" : 2,
            "outlettype" : [ "signal", "bang" ]
        }
    },
    {
        "box" : {
            "maxclass" : "newobj",
            "text" : "/ 100.",
            "patching_rect" : [ 892.857751071453094, 1235.514009118080139, 34.0, 20.0 ],
            "id" : "obj-123",
            "numoutlets" : 1,
            "numinlets" : 2,
            "outlettype" : [ "float" ]
        }
    },
    {
        "box" : {
            "maxclass" : "live.dial",
            "annotation" : "",
            "varname" : "StereoWidth",
            "prototypename" : "amount",
            "patching_rect" : [ 892.682948112487793, 1170.731735229492188,
87.19512403011322, 52.0 ],
            "presentation_rect" : [ 192.592589378356934, 106.481479704380035, 74.0, 52.0 ],

```



```

        "id" : "obj-122",
        "parameter_enable" : 1,
        "numoutlets" : 2,
        "fontsize" : 11.0,
        "numinlets" : 1,
        "presentation" : 1,
        "outlettype" : [ "", "float" ],
        "saved_attribute_attributes" : {
            "valueof" : {
                "parameter_initial" : [ 0 ],
                "parameter_initial_enable" : 1,
                "parameter_linknames" : 1,
                "parameter_longname" : "StereoWidth",
                "parameter_mmax" : 100.0,
                "parameter_modmode" : 0,
                "parameter_osc_name" : "<default>",
                "parameter_shortname" : "StereoWidth",
                "parameter_type" : 0,
                "parameter_unitstyle" : 5
            }
        }
    }
}

, {
    "box" : {
        "maxclass" : "newobj",
        "text" : "loadbang",
        "patching_rect" : [ 806.70733630657196, 1021.772136598825455, 53.0, 20.0 ],
        "id" : "obj-103",
        "numoutlets" : 1,
        "numinlets" : 1,
        "outlettype" : [ "bang" ]
    }
}

, {
    "box" : {
        "maxclass" : "newobj",
        "text" : "mc.+~",
        "patching_rect" : [ 826.512215256690979, 1385.365886688232422, 37.0, 20.0 ],
        "id" : "obj-99",
        "numoutlets" : 1,
        "numinlets" : 2,
        "outlettype" : [ "multichannelsignal" ]
    }
}

, {

```

```

    "box" :      {
      "maxclass" : "newobj",
      "text" : "mc.*~",
      "patching_rect" : [ 844.512215256690979, 1317.682958245277405, 35.0, 20.0 ],
      "id" : "obj-98",
      "numoutlets" : 1,
      "numinlets" : 2,
      "outlettype" : [ "multichannelsignal" ]
    }
  },
  {
    "box" :      {
      "maxclass" : "message",
      "text" : "deviate 0.5 0.",
      "patching_rect" : [ 844.512215256690979, 1115.243929028511047, 71.0, 20.0 ],
      "id" : "obj-97",
      "numoutlets" : 1,
      "numinlets" : 2,
      "outlettype" : [ "" ]
    }
  },
  {
    "box" :      {
      "maxclass" : "newobj",
      "text" : "mc.sig~ 0.5 @chans 100",
      "patching_rect" : [ 844.512215256690979, 1145.121978521347046, 124.0, 20.0 ],
      "id" : "obj-95",
      "numoutlets" : 1,
      "numinlets" : 1,
      "outlettype" : [ "multichannelsignal" ]
    }
  },
  {
    "box" :      {
      "maxclass" : "newobj",
      "text" : "mc.sig~ 0.5 @chans 100",
      "patching_rect" : [ 826.502611398696899, 1352.336438119411469, 124.0, 20.0 ],
      "id" : "obj-94",
      "numoutlets" : 1,
      "numinlets" : 1,
      "outlettype" : [ "multichannelsignal" ]
    }
  },
  {
    "box" :      {
      "maxclass" : "newobj",
      "text" : "mc.mixdown~ 2 @ autogain 1",

```

```

        "patching_rect" : [ 652.979248404502869, 1423.780521750450134,
192.523362994194031, 20.0 ],
        "id" : "obj-17",
        "numoutlets" : 1,
        "numinlets" : 2,
        "outlettype" : [ "multichannelsignal" ]
    }

},
    "lines" : [
        {
            "patchline" :
            {
                "source" : [ "obj-99", 0 ],
                "destination" : [ "obj-17", 1 ]
            }
        },
        {
            "patchline" :
            {
                "source" : [ "obj-98", 0 ],
                "destination" : [ "obj-99", 1 ]
            }
        },
        {
            "patchline" :
            {
                "source" : [ "obj-97", 0 ],
                "destination" : [ "obj-95", 0 ]
            }
        },
        {
            "patchline" :
            {
                "source" : [ "obj-95", 0 ],
                "destination" : [ "obj-98", 0 ]
            }
        },
        {
            "patchline" :
            {
                "source" : [ "obj-94", 0 ],
                "destination" : [ "obj-99", 0 ]
            }
        },
        {
            "patchline" :
            {
                "source" : [ "obj-4", 0 ],
                "destination" : [ "obj-97", 0 ]
            }
        }
    ]
}

```

```

    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-126", 0 ],
        "destination" : [ "obj-124", 0 ]
      }
    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-124", 0 ],
        "destination" : [ "obj-98", 1 ]
      }
    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-123", 0 ],
        "destination" : [ "obj-126", 0 ]
      }
    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-122", 0 ],
        "destination" : [ "obj-123", 0 ]
      }
    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-12", 0 ],
        "destination" : [ "obj-97", 0 ]
      }
    }
    ,
    {
      "patchline" :      {
        "source" : [ "obj-103", 0 ],
        "destination" : [ "obj-97", 0 ]
      }
    }
  ],
  "appversion" :  {
    "major" : 9,
    "minor" : 0,
    "revision" : 4,
    "architecture" : "x64",
    "modernui" : 1
  }
}

```

```
{
  "classnamespace": "box"
}
```

Dieser Patchteil in Max for Live (JSON-Auszug aus einem `.maxpat`) zeigt einen Abschnitt eines Multikanal-Audio-Patches mit Steuerung durch einen `live.dial`. Hier ist eine **funktionale Erklärung** der einzelnen Komponenten und wie sie zusammenspielen:

Funktionalität des Patchteils

Er **steuert die Breite eines Stereo-Effekts** (vermutlich durch Modulation oder Verteilung im Multichannel-Raum), wobei der Benutzer über einen Dial (`StereoWidth`) eine **Lautstärke-Skalierung** oder **Positionsvariation** beeinflussen kann.

Schritt-für-Schritt-Erklärung

1. `live.thisdevice`, `loadbang` und `button`

- `live.thisdevice` sendet beim Laden des Geräts einen `bang` aus.
- `loadbang` und ein `button` sind zusätzlich verdrahtet, um manuell oder automatisch beim Laden den nächsten Schritt auszulösen.
- Alle drei triggern die Nachricht:

```
csharp
[message] "deviate 0.5 0." → [mc.sig~ 0.5 @chans 100]
```

- Das bedeutet: **alle 100 Kanäle** des Signals erhalten eine **zufällige kleine Abweichung** um `0.5`.

2. `mc.sig~ 0.5 @chans 100` + `deviate 0.5 0.`

- `mc.sig~ 0.5` erzeugt für alle 100 Kanäle denselben Wert (`0.5`).
- Die Nachricht `"deviate 0.5 0."` bewirkt, dass **jedem Kanal** eine **zufällige Abweichung von max ± 0.5** zugewiesen wird.
- Dadurch entsteht **ein multikanales Signal mit leicht unterschiedlichem Pegel pro Kanal** → das kann zur **Stereoüberbreiterung** oder räumlichen Verteilung verwendet werden.

3. `live.dial` → Skalierungssignal

- Der User kann mit dem Dial `StereoWidth` (Werte von `0-100`) die Intensität der Modulation steuern.
- Die Werte gehen in:

CSS

```
[ / 100. ] → [ "$1 100" ] → [ line~ ]
```

Das ergibt z. B.:

arduino

```
70 → /100 → 0.7 → line~: 0.7 in 100ms
```

Also: **weich geglättete Steuerung** des Parameters mit ``line~``.

4. Signalverarbeitung: Skalieren & Addieren

- Das zufällig deviierte Signal (``mc.sig~ 0.5 + deviation``) wird mit dem ``line~``-Ausgang multipliziert:

CSS

```
[mc.sig~ + deviation] × [line~ output] → mc.*~
```

- Das Ergebnis wird dann **mit einem weiteren ``mc.sig~ 0.5`` addiert**, um eine Art **Offset oder Mittelwert** zu garantieren:

CSS

```
([mc.sig~ × control]) + 0.5
```

- Das erzeugt ein **leicht versetztes Stereo-Bild oder Panorama für jedes der 100 Kanäle**.

5. Ausgabe via ``mc.mixdown~ 2``

- Alle 100 Kanäle werden **in zwei Kanäle (Stereo)** heruntergemischt:

CSS

```
mc.mixdown~ 2 @autogain 1
```

- Das sorgt dafür, dass die 100 Kanäle nicht übersteuern und ordentlich summiert werden.

Ergebnis:

Das Patchsegment erzeugt ein **kontrolliert deviiertes multichannelsignal**, dessen "Breite" (Variation) über den ``StereoWidth``-Dial dynamisch verändert werden kann.

Das Ergebnis wird als **stereophonisches Audiosignal** ausgegeben.

Anwendungsidee:

Das kann benutzt werden für:

- **Stereo-Verbreiterung** (Breite eines Signals anpassen)
- **Multikanal-Spreading**
- **Randomisierte Lautstärken oder Parameter pro Kanal**
- Vorbereitung für spatialisiertes Routing oder Hallräume

Wenn du willst, kann ich dir das grafisch oder mit einem Testton visualisieren.